

A Comparative Study of Sensor-Based Path-Planning Algorithms in an Unknown Maze

Hiroshi Noborio, Keiichi Fujimura, Yohei Horiuchi

Department of Engineering Informatics
Osaka Electro-Communication University
Hatsu-Cho 18-8, Neyagawa, Osaka 572-8530, Japan
nobori@noblab.osakac.ac.jp

Abstract

In general, an unknown maze has few collision-free path to a destination. Therefore, if a robot supervised by the classic sensor-based path-planning algorithms Bug2, Class1, Alg1, Alg2 repeatedly enters into long local and global loops excluding and including a destination (goes out of its true way), respectively. For example, in Alg1 and Alg2, we can point out a case that a robot always enters into a global loop one time, and also in Bug(alter.) and Class1(alter.), we can find another case that a robot frequently joins a local loop many times. A complicated maze usually includes such cases, and therefore a robot arrives at a destination via a very long collision-free path. To overcome this, we revisit an algorithm HD - I whose following direction is adequately changed by the trial and error. In HD - I, a robot hardly selects an inadequate direction and consequently decreases a probability to enter into global and local loops.

1 Introduction

In the past decade, many sensor-based path-planning algorithms have been proposed [1]. In the sensor-based path-planning, a robot always selects one of two behaviors, that is, going straight to its destination and avoiding an encountered obstacle. The behaviors are switched if a robot hits or leaves an unknown obstacle. Here, as long as leave positions monotonically or asymptotically come close to a destination, convergence of a robot to its destination is completely ensured in an uncertain environment. This is called a metric condition. However, because of no environment information, a mobile robot supervised by almost all the classic algorithms unfortunately goes out of its way to a destination. Except for this, if each ob-

stacle has simple shape such as circle in an uncertain cluttered environment, we proposed a near-optimal sensor-based path-planning algorithm *Simple* and calculated a small competitive ratio between lengths of the optimal path and a selected path as 1.58 [2]. Otherwise, especially in an unknown huge maze, we have no proof to calculate a small competitive ratio and did not find a near-optimal sensor-based path-planning algorithm.

Many algorithms have been aggressively proposed for solving this problem. In the historical order, Lumelsky initially proposed *Bug1* and *Bug2* whose path lengths are denoted by $D + 1.5\sum_i P_i$ and $D + \sum_i \frac{n_i}{2} P_i$ on the worst, respectively [3] (D : the Euclidean distance between initial and target points, P_i : the perimeter of an i -th encountered obstacle, n_i : the number of encountering an i -th obstacle). However, many people guess that the latter algorithm selects a shorter collision-free path until a destination than the former algorithm on average. Therefore, *Bug2* has been repeatedly modified. For example, Sankaranarayanan modified *Bug2* as *Alg1* whose worst length is $D + 2\sum_i P_i$ by eliminating a repetitive loop (i.e., a side trip to a destination) [4]. On the other hand, one of the authors proposed *Class1* whose worst length is finitely bounded but is not evaluated, and also investigates convergence of a robot to its destination theoretically as the metric condition [5],[6]. Also, Sankaranarayanan modified *Class1* as *Alg2* whose worst length is $D + 2\sum_i P_i$ by eliminating a repetitive loop [7]. In general, we cannot determine whether *Class1* is superior to *Bug2* or not, whether *Alg1* is superior to *Bug2* or not, and whether *Alg2* is superior to *Class1* or not on average. For example, a path selected by *Bug2* and *Class1* is frequently shorter than that selected by *Alg1* and *Alg2*, respectively, in the same unknown environment (see examples in [8]). Recently, one of the authors proposed *Bug2(alter.)* and *Class1(alter.)*

(their original algorithms are called *Bug2(const.)* and *Class1(const.)*) [8]. The difference is for a robot to select alternatively clockwise and counter-clockwise directions following an uncertain obstacle. By the alternative walk, a robot quickly arrives at a destination, whose property is theoretically ensured on average. However, if a robot selects a bad direction as an initial direction following an uncertain obstacle, all the improved algorithms probably fail in seeking for a shorter collision-free path until a destination. This frequently occurs in a maze whose shape is complex.

Independently, one of the authors proposed a three or more dimensional sensor-based path-planning algorithm *HD - I* for a nonholonomic mobile robot and a robotic manipulator [9],[10],[11]. To follow a three or more dimensional obstacle, *HD - I* always learns many directions by trial and error and then finds a good direction for a robot to arrive at a destination. The learning is useful for selecting a shorter collision-free path in a huge uncertain maze because it prevents a robot from entering into a loop (i.e., a side trip to a destination).

The paper is organized as follows: Section 2 describes a robot and its unknown environment. In section 3, we describe the seven sensor-based path-planning algorithms *Bug2(const.)*, *Bug2(alter.)*, *Class1(const.)*, *Class1(alter.)*, *Alg1*, *Alg2* and *HD - I*. In section 4, we compare paths and lengths selected by these algorithms in two kinds of 2-D uncertain mazes and conclude *HD - I* is the best algorithm to select a shorter collision-free path until a destination on average. Finally, section 5 presents a few concluding remarks.

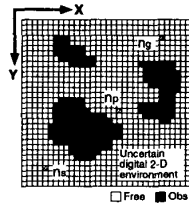


Figure 1 : A cell robot and its 2-D digital map with *Free* and *Obs* cells (nodes).

2 A Mobile Robot and Its Uncertain Environment

In this paper, we adopt a **cell decomposition approach** to path planning, which has a flexibility for

representing a complicated huge maze (Fig.1). First of all, a 2-D uncertain environment is defined by a digital map $[x_m, y_m](m = 1, \dots, 2^i)$. All the cells are built by dividing X and Y axes into an arbitrary resolution. According to the division, a digital map is built by 4^i cells (nodes). Then, all cells (nodes) are classified into two categories such as *Free* and *Obs* cells (nodes). If and only if a cell (node) does not intersect any obstacle, the cell (node) is *Free*, otherwise, the cell (node) is *Obs*. Secondly, a robot is expressed by a cell (node), and is able to move one of neighbor cells (nodes) in the digital map. In general, eight cells (nodes) are prepared for neighbors. Two neighbor cells (nodes) are implicitly connected by an edge. Furthermore, the distance (cost) of an edge between horizontal and vertical neighbors is defined by one, and the distance (cost) of an edge between diagonal neighbors is defined by a square route two ($\sqrt{2}$). A total distance between start and goal cells (nodes n_s and n_g) is calculated by summing these neighbor distances.

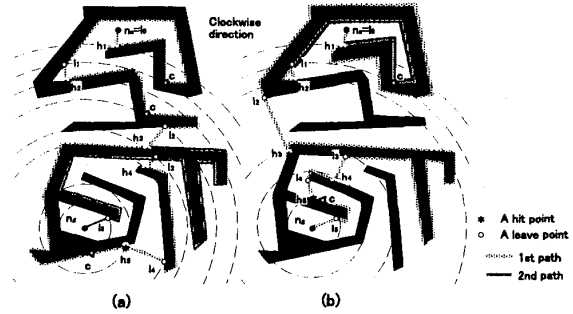


Figure 2 : (a) A collision-free path between n_s and n_g via many leave and hit points (h_j and l_j) selected in *Class1(const.)*. (b) Another path between n_s and n_g via many leave and hit points (h_j and l_j) selected in *Class1(alter.)*.

3 Seven Kinds of Algorithms

In this section, we briefly explain the sensor-based path-planning algorithms *Bug2(const.)*, *Bug2(alter.)*, *Class1(const.)*, *Class1(alter.)*, *Alg1*, *Alg2*, *HD - I* and their differences.

3.1 The algorithms *Class1(const.)* and *Class1(alter.)*

In this paragraph, we explain details of a classic algorithm *Class1(const.)* and its revised algorithm

Class1(alter.) (Fig.2) [5],[6].

1. Set $j = 1$, and set a robot node n_p and a leave node l_{j-1} and a closest node c_{j-1} by a start node n_s .

2. Select a neighbor node n_i of n_p , which the segment between centers of n_p and n_g intersects in a digital map.

(2a) If n_i is a goal node n_g , exit with success.

(2b) If n_i is *Free*, set n_p by n_i and then be continued, otherwise, set a hit node h_j and a closest node $c_j(j = j + 1)$ by n_p and go to step 3.

3. Select a neighbor *Free* node n_i of n_p by one of the clockwise and counter-clockwise orders based on the chain coding, which is adjacent to a neighbor *Obs* node until one of the following occurs. **The following order is fixed in advance in *Class1(const.)*, on the other hand, it is alternatively changed in *Class1(alter.)* (Fig.2).**

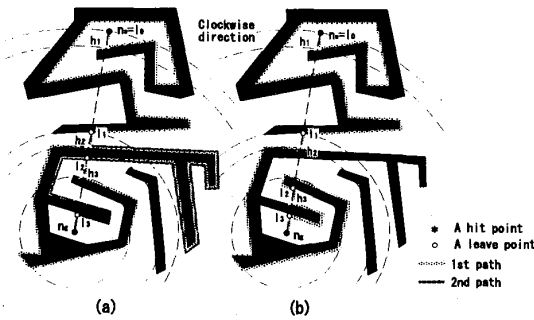


Figure 3 : (a) A collision-free path between n_s and n_g via many leave and hit points (h_j and l_j) selected in *Bug2(const.)*. (b) Another path between n_s and n_g via many leave and hit points (h_j and l_j) selected in *Bug2(alter.)*.

(3a) If n_i is a goal node n_g , exit with success.

(3b) If n_i is closer to n_g than the closest node c_j , reset c_j by n_i . Then, if the direction from n_i to n_g is *Free* (that is, a few nodes from n_i to n_g are *Free*), regard n_i as a leave node l_j and return to step 2.

(3c) If n_i equals to the last hit node h_j , exit with failure. In both algorithms *Class1(const.)* and *Class1(alter.)*, leave and hit nodes (l_j and h_j) monotonically come close to n_g , and consequently n_p arrives at n_g finally if a path (a sequence of *Free* nodes) between n_s and n_g exists in a given map. Otherwise, a point robot n_p automatically recognizes no solution (path) between n_s and n_g in an unknown map by returning the last hit node h_j without finding a leave point l_j .

3.2 The algorithms *Bug2(const.)* and *Bug2(alter.)*

In this paragraph, we briefly explain the difference between *Class1* and *Bug2*. The difference is the existence of the segment condition such that a robot n_i cannot leave an obstacle if it is not to be on the segment between centers of n_s and n_g (Fig.3). The details of *Bug2(const.)* and *Bug2(alter.)* are addressed in [3] and [8], respectively.

(3b) If n_i is to be on the segment between centers of n_p and n_g and also n_i is closer to n_g than c_j , a robot checks if the direction from n_i to n_g is *Free* (that is, a few nodes from n_i to n_g are *Free*). If it is right, regard n_i as a leave node l_j and return to step 2, otherwise, reset c_j by n_i .

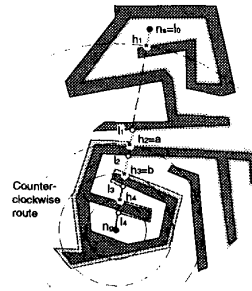


Figure 4 : A collision-free path between n_s and n_g via many leave and hit points (h_j and l_j) selected in *Alg1*.

3.3 The algorithm *Alg1*

In this paragraph, we briefly explain an algorithm *Alg1* (its detail is addressed in [4]) (Fig.4). The algorithm *Alg1* is designed under *Bug2(const.)* in order to get a shorter path until a destination frequently. Usually if a robot goes straight to a destination or follows an encountered obstacle one time, *Alg1* equals to *Bug2(const.)*. Otherwise, that is, if a robot meets a previous leave or hit point, the robot returns to the last hit point via a shorter route, and then reverses a direction following an obstacle. Then, after leaving the obstacle, *Alg1* equals to *Bug2(const.)*. As an example, when a mobile robot joins a cycle, e.g., a cycle from a to a via b , the robot backs to the last hit point $h_3(= b)$ by selecting the shorter route of clockwise and counter-clockwise ones (the counter-clockwise route is selected in Fig.4). Then, after backing to the last hit point $h_3(= b)$, the robot follows an unknown obstacle

in the opposite tracing order. Then after leaving the obstacle, $Alg1$ equals to $Bug2(const.)$.

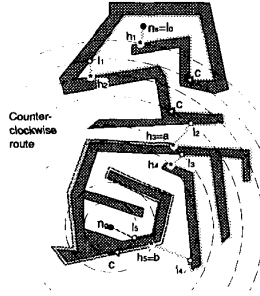


Figure 5 : A collision-free path between n_s and n_g via many leave and hit points (h_j and l_j) selected in $Alg2$.

3.4 The algorithm Alg2

In this paragraph, we briefly explain an algorithm $Alg2$ (its detail is addressed in [7]) (Fig.5). The algorithm $Alg2$ is designed under $Class1(const.)$ in order to get a shorter path until a destination frequently. The idea to shorten is just the same with $Alg1$. For example, when a mobile robot joins a cycle, e.g., a cycle from a to a via b , the robot backs to the last hit point $h_5(=b)$ by selecting the shorter route of clockwise and counter-clockwise ones (the counter-clockwise route is selected in Fig.5). Then, after backing to the last hit point $h_5(=b)$, the robot follows an unknown obstacle in the opposite tracing order. Then, after leaving the obstacle, $Alg2$ equals to $Class1(const.)$.

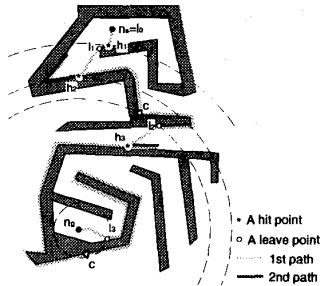


Figure 6 : A collision-free path between n_s and n_g via many leave and hit points (h_j and l_j) selected in $HD-I$.

3.5 The algorithm HD-I

In this paragraph, we focus on BF and $Class1$ which do not consider a **past** cost $g(n_p)$ from a start node n_s to a present node n_p . Then, we design a learning sensor-based path-planning algorithm $HD-I$ by mixing modified BF and $Class1$ (Fig.6) [9]. As mentioned previously, the sensor-based path-planning consists of two procedures. One procedure is for a robot to go straight to a goal. Another procedure is for a robot to avoid an uncertain obstacle. In the algorithm $HD-I$, the former procedure and the switching are designed by the modified $Class1$, and the latter procedure is designed by the modified BF .

(1) $HD-I$ uses BF in order for n_p to trace an encountered obstacle by clockwise and counter-clockwise orders. That is, if a robot node n_p meets a hit node h_j around an uncertain set of Obs nodes, BF starts (begins to input a found or expanded node into $OPEN$ or $CLOSED$ list), and if n_p reaches a leave node l_j around it, BF ends (cleans $OPEN$ and $CLOSED$ lists).

(2) $HD-I$ uses BF in order for n_p to trace boundary of an unknown obstacle, i.e., an uncertain set of Obs nodes. First of all, in a 2-D digital map, the number of nodes neighboring n_p is denoted by $3^2 - 1$. Secondly, the tracing starts if a robot node n_p finds a Obs node as its neighbor n_i on the segment between n_p and n_g . Therefore, n_p is *Free* and n_i is *Obs*. In this case, there is at least one *Free* node neighboring n_p , which neighbors the set of Obs nodes. All the *Free* nodes are inserted into $OPEN$.

(3) To select a hopeful node n^* from $OPEN$, we evaluate each node n^* by summing a **moving** cost $m(n^*)$ from n_p to n^* and a **future** cost $h(n^*)$ from n^* to n_g . The future cost is straightforwardly calculated by the *piecewise* L_2 metric along the segment between n^* and n_g . On the other hand, the moving cost is defined by multiplying α and the length of a truly traversed route between n_p and n^* .

1. Set $j = 1$, and set a robot node n_p and a leave node l_{j-1} and a closest node c_{j-1} by a start node n_s .

2. Select a neighbor node n_i of a robot node n_p , which the segment between n_p and n_g intersects in a digital map.

(2a) If n_i is a goal node n_g , exit with success.

(2b) If n_i is *Free*, set n_p by n_i and then be continued, otherwise, set a hit node h_j and a closest node $c_j(j = j + 1)$ by n_p and go to step 3.

3. Put the node h_j on a list called $OPEN$ of unexpanded nodes.

4. If $OPEN$ is empty, exit with failure; no path exists.

5. Remove from $OPEN$ a node n_p at which q is minimum (break ties arbitrary), and place it on a list called $CLOSED$ to be used for expanded nodes.

6. Select each neighbor *Free* node n_i of n_p , which neighbors on the set of Obs nodes. For every neighbor n_i :

(6a) If n_i is a goal node n_g , exit successfully.

		Class1		Bug2		Alg		HD-I
		const.	alter.	const.	alter.	1	2	$\alpha = 0.5$
$n_s:(7,7)-n_g:(277,255)$	clockwise	11996	1574	20847	18088	20847	11996	6515
	c-clockwise	24933	1640	long	18052	16187	28987	—
$n_s:(506,7)-n_g:(277,255)$	clockwise	13601	long	7376	25783	7376	8014	9473
	c-clockwise	5255	long	7418	25746	7417	5255	—
$n_s:(506,506)-n_g:(277,255)$	clockwise	4466	19825	12307	long	12307	3901	4222
	c-clockwise	14301	19809	13470	long	13284	10498	—
$n_s:(7,506)-n_g:(277,255)$	clockwise	19196	long	5268	long	5268	4385	6383
	c-clockwise	25233	long	22666	long	22666	23454	—

Table 1 : Path lengths from four start positions n_s to a goal position n_g in *Class1*, *Bug2*, *Alg1*, *Alg2*, and *HD-I* in a 2-D uncertain maze.

(6b) If n_i is closer to n_g than the closest node c_j , reset c_j by n_i , and if a few nodes from n_i to n_g are *Free*, regard n_i as a leave node l_j . Then, clear *OPEN* and *CLOSED* and return to step 2.

(6c) If n_i was neither on *OPEN* nor on *CLOSED*, register it to *OPEN* and generate n_i with the pointers back to n_p .

(6d) Evaluate each node n^* in *OPEN* as follows: Firstly, calculate a future cost $h(n^*)$ by the *piecewise* L_2 metric along the segment between n^* and n_g . Secondly, calculate a moving cost $m(n^*)$ by the *piecewise* L_2 metric along a route already traveled between n_p and n^* . Finally, assign $q(n^*) = \alpha \times m(n^*) + h(n^*)$ ($0 < \alpha < 1$) to each node n^* in *OPEN*, and then return to step 4.

In the algorithm *HD-I*, leave and hit nodes (l_j and h_j) monotonically converge to a goal node (n_g) in a 2-D digital map. In addition, n_p always finds an adequate leave node l_j around an encountered set of *Obs* nodes as long as a sequence of *Free* nodes from n_s to n_g exists in a given digital map. For this reason, if such a sequence exists, n_p certainly arrives at n_g , otherwise, n_p sees no path since *OPEN* becomes empty.

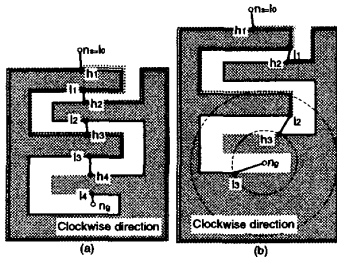


Figure 7 : (a),(b) A mobile robot joins similar global loops several times and extremely increases a driving distance to a destination in *Bug2(const.)* and *Class1(const.)*, respectively.

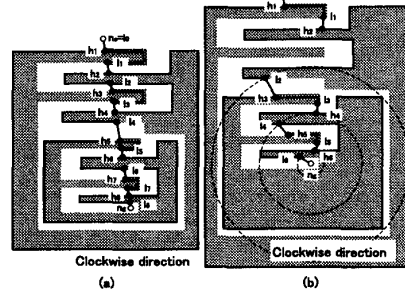


Figure 8 : (a),(b) A mobile robot joins a global loop one time and consequently does not arrive at a destination quickly in *Alg1* and *Alg2*, respectively.

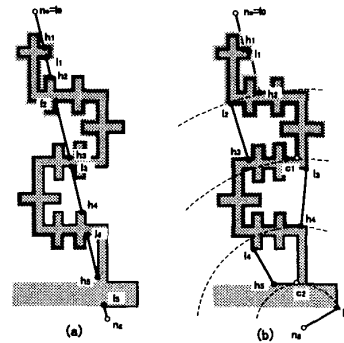


Figure 9 : (a),(b) A mobile robot joins a local loop several times and consequently its collision-free path becomes very long in *Bug2(alter.)* and *Class1(alter.)*, respectively.

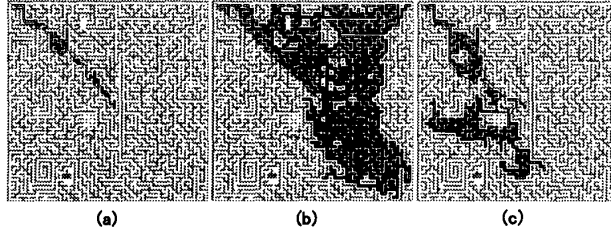


Figure 12 : (a) A collision-free path from $n_s:(7,7)$ to $n_g:(277,255)$ selected by *Class1(alter.)[clockwise]* in a 2-D uncertain maze. (b) A path between the same pair selected by *Alg1[clockwise]* in the maze. (c) A path between the same pair selected by *HD - I* in the maze.

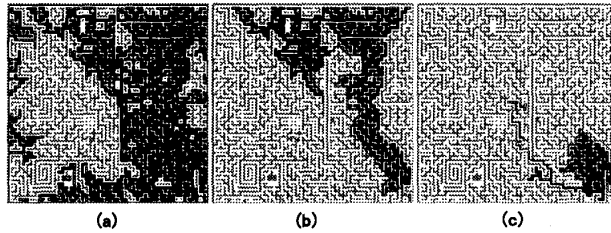


Figure 13 : (a) A collision-free path from $n_s:(506,506)$ to $n_g:(277,255)$ selected by *Bug2(alter.)[c - clockwise]* in a 2-D unknown maze. (b) A path between the same pair selected by *Alg1[c - clockwise]* in the maze. (c) A path between the same pair selected by *HD - I* in the maze.

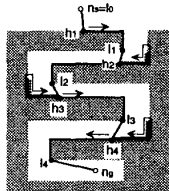


Figure 10 : A mobile robot does not easily join a global loop and consequently arrives at a destination quickly in *HD - I*.

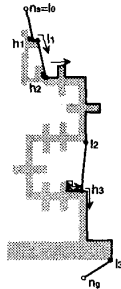


Figure 11 : A mobile robot does not easily join a local loop and consequently arrives at a destination quickly in *HD - I*.

4 Discussions

In an unknown maze, a robot enters into global and local loops several times and consequently generates a very long path until a destination. For example, the most classic algorithms *Bug2(const.)* and *Class1(const.)* pick up very long collision-free paths in an uncertain environment (Fig.7). A few of classic sensor-based path-planning algorithms partially overcome this problem as follows: In the algorithms *Alg1* and *Alg2*, a robot returns to the closest position to a destination and reverses its direction following an obstacle (i.e., escapes from a global or local loop) if the robot encounters a position visited already (i.e., meets the loop). By the procedure, a robot never enters into the same loop after this. In the algorithms *Bug2(alter.)* and *Class1(alter.)*, a robot alternatively changes a direction following an encountered obstacle. This theoretically brings decreasing a probability that a robot joins a global loop. Moreover, in *Single*, we succeeded in mixing advantages of *Alg1* or *Alg2* and *Bug2(alter.)* or *Class1(alter.)* in an uncertain maze with a unique collision-free path to a destination [12].

However, because of a contradiction, we did not succeed in mixing advantages of *Alg1* or *Alg2* and *Bug2(alter.)* or *Class1(alter.)* in an uncertain maze with two or more collision-free paths. In general, a weak point of *Alg1* or *Alg2* is for a robot to enter into a global loop one time (Fig.8), on the other hand, a weak point of *Bug2(alter.)* or *Class1(alter.)* is for a

		Class1		Bug2		Alg		HD-I
		const.	alter.	const.	alter.	1	2	$\alpha = 0.5$
$n_s:(9,9)-n_g:(275,263)$	clockwise	long	long	long	11027	27579	22023	2739
	c-clockwise	long	long	long	10690	10668	11628	—
$n_s:(502,9)-n_g:(275,263)$	clockwise	long	long	long	long	18368	14498	3627
	c-clockwise	18578	long	15676	long	5356	4029	—
$n_s:(502,502)-n_g:(275,263)$	clockwise	long	long	21626	long	21626	20372	15426
	c-clockwise	15566	29486	6027	long	6027	21633	—
$n_s:(9,502)-n_g:(275,263)$	clockwise	26384	8080	long	long	6308	5453	1433
	c-clockwise	9735	7960	16769	long	8063	6015	—

Table 2 : Path lengths from four n_s to one n_g in Class1, Bug2, Alg1, Alg2, and HD - I in another 2-D uncertain maze.

robot to enter into a local loop several times (Fig.9).

As contrasted with this, we proposed a sensor-based path-planning algorithm *HD - I* whose trial and error learns a hopeful direction following an encountered obstacle. In order to find a leave position around an unknown object, the algorithm *HD - I* uses a modified version of the famous *Best - First* algorithm. The reason why we adopt *BF* is the similarity to the sensor-based path-planning algorithm with the metric condition. That is, both do not consider any cost from a start position to a present position. In other words, both consider only a future cost from a present position to a target position, which is called **heuristics**. The modified algorithm *BF* always selects a most hopeful point around an encountered obstacle. This trial and error flexibly changes a direction to find an adequate leave point around an obstacle. This probably overcomes the previous weak points because the trial and error does not allow a robot to go away from a destination (Fig.10 and 11).

5 Simulation Results

In this section, we compare paths and lengths selected by typical sensor-based path-planning algorithms *Bug2*, *Class1*, *Alg1*, *Alg2*, and *HD - I* in two kinds of uncertain huge mazes and then select the best algorithm. A 2-D digital map [x,y] is defined as Fig.1, and many obstacles whose shape is complex are colored by gray, and a selected path is colored by black.

Firstly, a huge uncertain maze is randomly constructed, and then three different collision-free paths are selected from $n_s:(7,7)$ to $n_g:(277,255)$ by *Class1* (*alter.*) [clockwise], *Alg1* [clockwise] and *HD - I* in an uncertain maze described at Fig.12(a),(b) and (c), respectively. In addition, three different collision-free paths are selected from $n_s:(506,506)$ to $n_g:(277,255)$ by *Bug2*(*alter.*) [c-clockwise], *Alg1* [c-clockwise] and *HD - I* in the same maze shown at Fig.13(a),(b) and (c), respectively. Furthermore, if we change n_s , we obtain the other results (Table 1, *long* means path length is over 30000). In the results, our algorithm *HD - I* selects the shortest or shorter path. Especially, path lengths of all the previous algorithms except for *HD - I* extremely depend on an initial direction fol-

lowing an encountered obstacle. However, *HD - I* always learns an adequate direction by the trial and error, and therefore selects a shorter collision-free path until a destination, which has no connection with an initial direction.

Secondly, another huge uncertain maze is built at random and then three different collision-free paths are picked up from $n_s:(9,502)$ to $n_g:(275,263)$ by *Class1* (*alter.*) [clockwise], *Alg2* [clockwise] and *HD - I* in another unknown maze illustrated at Fig.14(a),(b) and (c), respectively. In addition, three different collision-free paths are selected from $n_s:(502,9)$ to $n_g:(275,263)$ by *Bug2*(*const.*) [c-clockwise], *Alg1* [c-clockwise] and *HD - I* in the same maze shown at Fig.15(a),(b) and (c), respectively. Furthermore, by changing n_s , we obtain the other results (Table 2, *long* means path length is over 30000). In the rest three results, our algorithm *HD - I* always learns an adequate direction by the trial and error, and therefore selects a shorter collision-free path until a destination.

As a result, on the average, our learning sensor-based path-planning algorithm *HD - I* catches the shortest or a shorter path in a huge uncertain maze whose shape is complicated. Finally, all programs are running in a personal computer Gateway GP6-400 (Windows 98 with Open GL, Pentium II 400MHz, 256MB main memory).

6 Conclusions

In this paper, we compared typical seven sensor-based path-planning algorithms in two kinds of uncertain mazes, and regarded the algorithm *HD - I* as the best on the average. This was ascertained by theoretical discussions and simulation results. In future, we will be seeking for the algorithm whose superiority is theoretically ensured.

References

- [1] H.Noborio, "On a sensor-based navigation for a mobile robot," *Journal of Robotics Mechatronics*, Vol.8, No.1, pp.2-14, 1996.

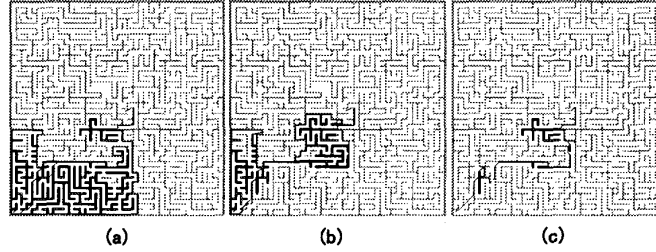


Figure 14 : (a) A collision-free path from $n_s:(9,502)$ to $n_g:(275,263)$ selected by *Class1(alter.)[clockwise]* in another 2-D unknown maze. (b) A path between the same pair selected by *Alg2[clockwise]* in the maze. (c) A path between the same pair selected by *HD-I* in the maze.

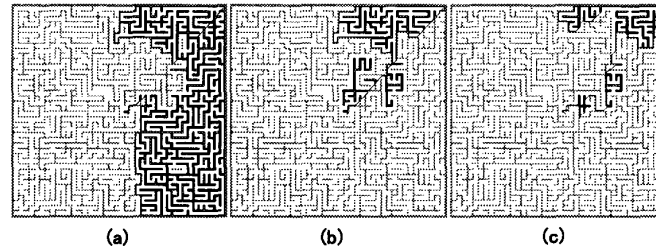


Figure 15 : (a) A collision-free path from $n_s:(502,9)$ to $n_g:(275,263)$ selected by *Bug2(const.)[c-clockwise]* in another 2-D unknown maze. (b) A path between the same pair selected by *Alg1[c-clockwise]* in the maze. (c) A path between the same pair selected by *HD-I* in the maze.

- [2] Hiroshi Noborio and Kenji Urakawa, "A near-optimal sensor-based navigation among 2-D uncertain obstacles with simple shape," *Proc. of the 1999 IEEE Int. Conf. on Robotics and Automation*, pp.355-360, April 1999.
- [3] V.J.Lumelsky and A.A.Stepanov, "Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape," *Algorithmica*, Vol.2, pp.403-430, 1987.
- [4] A.Sankaranarayanan and M.Vidyasager, "A new path planning algorithm for moving a point object amidst unknown obstacles in a plane," *Proc. of the 1990 IEEE Int. Conf. on Robotics and Automation*, pp.1930-1936, May 1990.
- [5] H.Noborio, "A path-planning algorithm for generation of an intuitively reasonable path in an uncertain 2D workspace", *Proc. of the Japan-USA Symposium on Flexible Automation*, pp.477-480, July 1990.
- [6] H.Noborio, "A sufficient condition for designing a family of sensor-based deadlock-free path-planning algorithms," *Journal of Advanced Robotics*, Vol.7, No.5, pp.413-433, January 1993.
- [7] A.Sankaranarayanan and M.Vidyasagar, "Path planning for moving a point object amidst unknown obstacles in a plane: a new algorithm and a general theory for algorithm development," *Proc. of the 29th Conf. on Decision and Control*, pp.1111-1119, December, 1990.
- [8] H.Noborio, T.Yoshioka, and S.Tominaga, "On the sensor-based navigation by changing a direction to follow an encountered obstacle," *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp.510-517, September 1997.
- [9] H.Noborio, Y.Maeda, and K.Urakawa, "Three or more dimensional sensor-based path-planning algorithm HD-I," *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp.1699-1706, October 1999.
- [10] H.Noborio, I.Yamamoto, T.Komaki, "Sensor-based path-planning algorithms for a nonholonomic mobile robot", *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, October 2000 (to appear).
- [11] J. Pearl, "Heuristics", *Addison Wesley*, October 1985.
- [12] H.Noborio, M.Kadowaki, and K.Urakawa, "A near-optimal sensor-based motion-planning algorithm for parts mating," *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, pp.510-517, October 1998.
- [13] S.Sundar and Z.Shiller, "Optimal obstacle avoidance based on the hamilton-jacobi-bellman equation," *IEEE Trans. on Robotics and Automation*, Vol.13, No.2, pp.305-310, 1997.
- [14] I.Kamon and E.Rivlin, "Sensory-based motion planning with global proofs," *IEEE Trans. on Robotics and Automation*, Vol.13, No.6, pp.814-822, 1997.